

Fully Convolutional Networks for Semantic Segmentation

Jonathan Long¹, Evan Shelhamer¹, Trevor Darrell¹

¹UC Berkeley

Convolutional Networks (ConvNets or CNNs) are a type of neural network designed to process images by breaking them into small parts using filters (or kernels). Traditional ConvNets are usually used to classify images as a whole by outputting just a single label that represents the content of the whole image.

On the other hand, FCNs are an extension of traditional CNNs, where instead of producing a single output for the entire image, they make predictions for each individual pixel, making them ideal for tasks like semantic segmentation, where the goal is to classify each pixel in an image. Also, FCNs can take an image of any size as input and output a corresponding dense prediction, which is a prediction for every pixel in the image.

In the paper[3], the authors state that this was the first work to train FCNs end-to-end for pixel-wise prediction and to do so using supervised pre-training. Here, end-to-end training means that the network is trained in one go from input to output rather than being trained on patches or requiring separate steps, which were the methods commonly used in previous models. Additionally, supervised pre-training refers to training the network on a large labeled dataset first, and then fine-tuning it for the specific task of semantic segmentation. The authors also compared their FCN approach with past methods, demonstrating superior performance on semantic segmentation benchmarks like PASCAL VOC and NYUDv2. Upon doing so, it was found that FCNs eliminate the need for additional techniques like super-pixel projection and random field regularization, achieving state-of-the-art results.

CNNs operate on three-dimensional data (height h , width w , and depth d) where h and w are spatial dimensions and d is the channel dimension (e.g., color channels). meaning their components (convolution, pooling, and activation functions) depend on relative spatial coordinates. The output y_{ij} at location (i, j) in a layer is computed by:

$$y_{ij} = f_{ks} \left(\{x_{si+\delta i, sj+\delta j}\}_{0 \leq \delta, \delta_j \leq k} \right)$$

where k is the kernel size, s is the subsampling factor, and $f_{k,s}$ determines the layer type (e.g., convolution, pooling, or activation function). The output y_{ij} at location (i, j) in a layer is computed by applying the function $f_{k,s}$, which depends on the input data vector x_{ij} and a local region defined by a kernel of size k and stride s . This process is used to compute activations in the network.

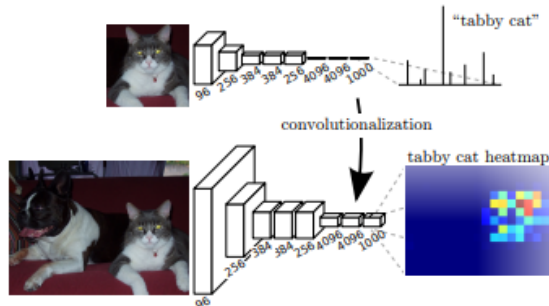


Figure 1: Transforming fully connected layers into convolution layers enables a classification net to output a heatmap. Adding layers and a spatial loss produces an efficient machine for end-to-end dense learning.

The transformation of traditional recognition networks like LeNet[2] and AlexNet[1] into FCNs is also discussed in the paper. According to the authors, these recognition networks use fully connected layers that require fixed input sizes and generate nonspatial outputs, discarding spatial information. However, by reinterpreting these fully connected layers as convolutions that span the entire input region, the network becomes a fully

convolutional one, capable of handling inputs of any size and producing classification maps. This transformation, as shown in Figure 2, improves computational efficiency.

Shift-and-Stitch, Upsampling, and Patchwise Training are key techniques for generating dense predictions in FCNs. The Shift-and-Stitch method involves shifting the input by small pixel amounts, running these shifted inputs through the network, and then combining the outputs to create dense predictions without needing interpolation. While this method enhances output density, it limits the filters' ability to access finer details. Upsampling, on the other hand, involves backwards strided convolution (deconvolution) with a fractional stride to upscale coarse outputs into dense, finer predictions. This method is said to be more effective than shift-and-stitch in the paper. Additionally, Patchwise Training is another technique discussed. This method involves selecting random patches from the image during training, which helps reduce the computational burden compared to training on full images. However, the authors found that whole-image training, which involves training on the full image rather than patches, is more efficient and effective for dense prediction tasks. Combining techniques like upsampling with skip layer fusion has proven to be more successful for learning dense predictions in FCNs.

The adaptation of ILSVRC classifiers like AlexNet[1], VGG[4], and GoogLeNet[5] into FCNs for semantic segmentation was also mentioned. The experiment converting the fully connected layers of these classifiers into convolutional layers and fine-tuning them for segmentation. The authors later introduce a novel skip architecture that combines coarse, semantic, and local appearance information to refine predictions. The architecture uses layers with different strides and combines them through upsampling to improve spatial precision in the output. The FCN-16s model, which fuses outputs from lower and higher layers, improves the mean Intersection over Union (IU) to 62.4 on the PASCAL VOC 2011 validation set. Further refinement is achieved by fusing additional layers to create FCN-8s, achieving a slight improvement to 62.7 IU. The authors explore other methods for finer predictions, like decreasing pooling strides and using the shift-and-stitch trick, but find that layer fusion offers the best trade-off in terms of performance and computational cost.

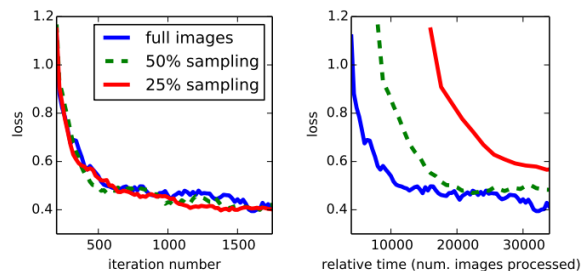


Figure 2: Training on whole images is just as effective as sampling patches, but results in faster (wall time) convergence by making more efficient use of data. Left shows the effect of sampling on convergence rate for a fixed expected batch size, while right plots the same by relative wall time

The authors also trained the networks using SGD with momentum and fine-tune all layers by backpropagating through the entire network. They find that fine-tuning the full network results in better performance than only fine-tuning the classifier. Figure 2 compares the convergence rate of training on full images versus using different levels of sampling, showing that whole-image training is just as effective but leads to faster convergence in terms of wall time.

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, pages 1–5, 2012.
- [2] Y. LeCun, B. Boser, J. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to hand-written zip code recognition. *Neural Computation*, 1(4):541–551, 1989.
- [3] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation, 2015. URL <https://arxiv.org/abs/1411.4038>.
- [4] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *CoRR*, abs/1409.1556, 2014.
- [5] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. *CoRR*, abs/1409.4842, 2014.